INSIDE STLC ACADEMY

QA LEADERSHIP ACADEMY

Automation Assessment

A structured health-check of an existing automation suite before you invest another penny in it.

Purpose

This instrument gives you an honest, evidence-based picture of an inherited automation suite: what it costs, what it is worth, and whether it should be extended, stabilised, re-platformed or retired. It turns "the automation is flaky" into a defensible position you can take to your VP Engineering.

It scores a suite across seven dimensions — coverage, reliability, speed, maintainability, ownership, trust and value — so that a decision to invest (or stop investing) is grounded in the current state rather than in enthusiasm or frustration.

When to use it

- In your first 30–60 days in a QA leadership role, to baseline what you have inherited.
- When leadership is pushing for "more automation" without agreeing what problem it solves.
- Before approving a re-platform, a tooling change or a business case for automation headcount.
- Annually, or after a major architecture change, as a portfolio review.

Instructions

1. Gather the evidence first: run history, average duration, flake/quarantine counts, who last touched the code, and where the tests sit in the pyramid (unit / API / UI).
2. Score each of the seven dimensions from 1 (poor) to 5 (strong) using the anchor descriptions, and write one sentence of evidence for each score — never a bare number.
3. Multiply each score by its weight and total the weighted column to get an overall health score out of 100.
4. Read the score band, then decide a direction of travel for the suite (stabilise / extend / re-platform / retire) rather than a to-do list.
5. Re-run the assessment on the same dimensions each quarter so the trend, not the snapshot, drives the conversation.

Worked example

Northstar Digital has ~1,800 Selenium UI tests, a ~6-hour run and roughly 25% flakiness, owned by QA alone with developers not touching it and trust described as low. Scored against the model:

Dimension	Weight	Score (1–5)	Evidence	Weighted
Coverage vs risk	20	3	Broad UI coverage but thin at API/unit; critical payment paths only exercised end-to-end	12
Reliability (flake rate)	20	1	~25% flaky; results routinely re-run or ignored	4
Feedback speed	15	1	~6-hour run; cannot gate a pull request or a same-day release	3
Maintainability	15	2	One engineer (Dan) holds all context; no page-object discipline agreed	6
Ownership model	10	1	QA-only; developers do not read, write or fix the tests	2
Trust / usage	10	1	Low trust; squads route around it rather than block on it	2
Business value delivered	10	2	Rarely credited with catching an escaped defect; value is asserted, not shown	4

Weighted total: 33 / 100 — a low-trust, high-cost suite. The reading is not "add more tests"; it is "the current suite is a liability being propped up by one person". A sensible direction of travel: quarantine the flakiest ~25% to restore a trustworthy signal, push coverage down the pyramid to API level for the payment paths, and introduce shared ownership so Dan is not the single point of failure.

> **READ THE SCORE AS A DIRECTION, NOT A VERDICT**
> A score of 33/100 does not mean "delete the suite". It means the fastest route to value is stability and trust before volume. Present it as a direction of travel leadership can fund in stages.

Blank reusable version

Dimension	Weight	Score (1–5)	Evidence (one sentence)	Weighted
Coverage vs risk	20			
Reliability (flake rate)	20			
Feedback speed	15			
Maintainability	15			
Ownership model	10			
Trust / usage	10			
Business value delivered	10			
TOTAL	100	—	—	

Score bands

- 75–100 — Healthy: extend deliberately against risk; protect what works.

- 50–74 — Serviceable: targeted improvement on the two weakest dimensions.
- 25–49 — Fragile: stabilise and rebuild trust before adding volume.
- Below 25 — Liability: consider re-platform or retire; stop adding tests to it.

Common mistakes

COMMON MISTAKES

Scoring on gut feel with no evidence sentence; treating a low score as licence to delete everything overnight; assessing volume ("how many tests") instead of value and trust; and running it once as a stick to beat the previous owner with, rather than quarterly to show a trend.

How to interpret the results

- The two lowest-weighted-scoring dimensions are your first investment, not the highest-coverage gap.
- A low reliability score caps the value of everything else — an untrusted suite delivers little regardless of coverage.
- A single-owner maintainability score is a key-person risk to name explicitly to leadership.
- Compare quarter on quarter: a rising trend from a low base is a stronger story than a static high score.

INSIDE STLC PRO TIP

Walk into the VP Engineering conversation with the weighted score and one evidence sentence per row. "Our UI suite scores 33/100, capped by 25% flakiness and single-person ownership" reframes the debate from "we need more automation" to "we need trustworthy automation first" — and that is a business case you can actually win.